

TP Chapitre 2 : Organisation Mémoire

Exploration Pratique avec GDB et Valgrind

Dr. El Hadji Bassirou TOURÉ

Département de Mathématiques et Informatique

Université Cheikh Anta Diop de Dakar

Prérequis

Ce TP suppose que vous avez configuré votre environnement Docker (TP-0) et maîtrisez :

- Compilation avec gcc et options -Wall -Wextra -g
- Débogage de base avec gdb
- Installation de valgrind : apt-get install valgrind

Commandes de référence

```
# Compilation avec symboles de debug
gcc -Wall -Wextra -g prog.c -o prog

# Compilation avec Address Sanitizer
gcc -Wall -Wextra -g -fsanitize=address prog.c -o prog

# Débogage mémoire avec GDB
gdb ./prog
(gdb) break main
(gdb) run
(gdb) print &variable          # Adresse
(gdb) print/x variable         # Hexadécimal
(gdb) x/4xb &variable          # Examiner 4 octets en hex
(gdb) x/s str                   # Examiner comme string

# Détection de fuites mémoire
valgrind --leak-check=full ./prog
```

1 Exercice 1 : Visualisation Mémoire et Endianness

Objectif

Examiner la représentation mémoire des variables avec GDB et vérifier l'endianness.

1.1 Créer un fichier memory_view.c :

```
1 #include <stdio.h>
2 #include <stdint.h>
3
4 int main() {
5     int x = 0x12345678;
6     short s = 0xABCD;
7     char c = 'A';
8
9     printf("x = 0x%X, &x = %p\n", x, (void*)&x);
10    printf("s = 0x%X, &s = %p\n", s, (void*)&s);
11    printf("c = '%c' (0x%X), &c = %p\n", c, c, (void*)&c);
12
13    return 0;
14 }
```

Questions :

- a) Compiler avec gcc -Wall -g memory_view.c -o memory_view.

- b) Lancer GDB et mettre un breakpoint après les déclarations.
- c) Utiliser `x/4xb &x` pour afficher les 4 octets de `x`. Dans quel ordre sont-ils stockés ?
- d) Vérifier que votre machine est bien little-endian.
- e) Utiliser `x/2xb &s` pour afficher les octets de `s`. Vérifier l'ordre.

1.2 Ajouter au programme :

```

1  int arr[3] = {0x0A, 0x14, 0x1E};
2  int *p = arr;
3  int *q = &arr[2];
4
5  printf("\n=== Arithmétique des pointeurs ===\n");
6  printf("p = %p, q = %p\n", (void*)p, (void*)q);
7  printf("q - p = %ld (éléments)\n", q - p);
8  printf("(char*)q - (char*)p = %ld (octets)\n", (char*)q - (char*)p);

```

Questions :

- f) En GDB, utiliser `x/12xb arr` pour voir les 12 octets du tableau.
- g) Vérifier que `q - p = 2` (éléments) et non 8 (octets).
- h) Calculer manuellement `&arr[1]` à partir de `&arr[0]`.

2 Exercice 2 : Débordement de Tableau (Buffer Overflow)

Objectif

Observer les conséquences d'un accès hors limites et utiliser Address Sanitizer.

Attention

Ce code contient des bugs intentionnels ! Le but est de les détecter.

2.1 Créer un fichier `overflow.c` :

```

1  #include <stdio.h>
2
3  int main() {
4      int secret = 0xDEADBEEF;
5      int arr[4] = {1, 2, 3, 4};
6      int guard = 0xCAFEBADE;
7
8      printf("Avant overflow:\n");
9      printf("secret = 0x%X\n", secret);
10     printf("guard = 0x%X\n", guard);
11
12     printf("\nAdresses:\n");
13     printf("&secret = %p\n", (void*)&secret);
14     printf("&arr[0] = %p\n", (void*)&arr[0]);
15     printf("&arr[4] = %p (hors limites!)\n", (void*)&arr[4]);
16     printf("&guard = %p\n", (void*)&guard);
17
18     // OVERFLOW INTENTIONNEL !
19     printf("\n critique arr[4] = 0xBAD...\n");
20     arr[4] = 0xBAD; // Hors limites !
21
22     printf("\nAprès overflow:\n");
23     printf("secret = 0x%X\n", secret);
24     printf("guard = 0x%X\n", guard);
25
26     return 0;
27 }

```

Questions :

- a) Compiler SANS sanitizer : `gcc -g overflow.c -o overflow`. Exécuter. Quelle variable a été corrompue ?
- b) Compiler AVEC Address Sanitizer : `gcc -g -fsanitize=address overflow.c -o overflow`. Que se passe-t-il ?
- c) Tester aussi `arr[-1] = 0xBAD`. Quelle variable est affectée ?
- d) Expliquer pourquoi le compilateur ne détecte pas ces erreurs à la compilation.

3 Exercice 3 : Chaînes et Allocation Dynamique

Objectif

Comprendre les chaînes en mémoire et détecter les fuites avec Valgrind.

3.1 Créer un fichier strings_alloc.c :

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 int main() {
6     // Partie 1: Chaînes
7     char arr[] = "Hello";
8     char *ptr = "World";
9
10    printf("=== Chaînes ===\n");
11    printf("arr = \"%s\", sizeof = %zu, strlen = %zu\n",
12          arr, sizeof(arr), strlen(arr));
13    printf("ptr = \"%s\", sizeof = %zu, strlen = %zu\n",
14          ptr, sizeof(ptr), strlen(ptr));
15
16    // Modification
17    arr[0] = 'J'; // OK
18    printf("Après arr[0]='J': %s\n", arr);
19    // ptr[0] = 'X'; // CRASH ! Décommenter pour tester
20
21    // Partie 2: Allocation dynamique
22    printf("\n=== Allocation dynamique ===\n");
23    int *a = malloc(5 * sizeof(int));
24    int *b = calloc(5, sizeof(int));
25
26    printf("malloc: a[0] = %d (non initialisé!)\n", a[0]);
27    printf("calloc: b[0] = %d (initialisé à 0)\n", b[0]);
28
29    for (int i = 0; i < 5; i++) {
30        a[i] = i * 10;
31    }
32
33    // FUITE INTENTIONNELLE : b n'est pas libéré !
34    free(a);
35    // free(b); // Décommenter pour corriger
36
37    return 0;
38 }
```

Questions :

- Compiler et exécuter. Pourquoi `sizeof(arr) = 6` mais `sizeof(ptr) = 8` ?
- Décommenter `ptr[0] = 'X'`. Compiler et exécuter. Que se passe-t-il ? Pourquoi ?
- Exécuter avec Valgrind : `valgrind -leak-check=full ./strings_alloc`. Combien d'octets sont perdus ?
- Corriger la fuite en ajoutant `free(b)`. Relancer Valgrind pour vérifier.
- En GDB, utiliser `x/6cb arr` pour afficher les caractères. Identifier le `'\0'` final.

4 Exercice 4 : Projet — Visualiseur Hexadécimal

Objectif

Créer un outil pour afficher un dump hexadécimal de n'importe quelle variable.

4.1 Créer un fichier hexdump.c :

```
1 #include <stdio.h>
2 #include <stdint.h>
3
4 // Affiche 'size' octets à partir de 'ptr'
5 void hexdump(const char *name, void *ptr, size_t size) {
6     unsigned char *p = (unsigned char *)ptr;
7     printf("=== %s (%zu octets à %p) ===\n", name, size, ptr);
8     for (size_t i = 0; i < size; i++) {
9         if (i % 8 == 0 && i > 0) printf("\n");
10        printf("%02X ", p[i]);
11    }
12    printf("\n\n");
13 }
14
15 int main() {
16     int x = 0x12345678;
17     hexdump("int x = 0x12345678", &x, sizeof(x));
18
19     short s = 0xABCD;
```

```

20     hexdump("short s = 0xABCD", &s, sizeof(s));
21
22     char str[] = "UCAD";
23     hexdump("char str[] = \"UCAD\\\"", str, sizeof(str));
24
25     int arr[3] = {10, 20, 30};
26     hexdump("int arr[3] = {10, 20, 30}", arr, sizeof(arr));
27
28     // TODO: Ajouter un test avec un pointeur
29     int *p = &x;
30     hexdump("int *p = &x", &p, sizeof(p));
31
32     return 0;
33 }

```

Questions :

- Compiler et exécuter. Vérifier que 0x12345678 apparaît comme 78 56 34 12 (little-endian).
- Pour la chaîne "UCAD", identifier les codes ASCII et le terminateur 00.
- Pour le pointeur p, vérifier que sa valeur correspond bien à &x.
- Améliorer hexdump pour afficher aussi les caractères ASCII imprimables à droite (comme la commande xxd).

5 Soumission

À rendre

- Tous les fichiers source (.c)
- Un fichier README.md avec :
 - Réponses aux questions de chaque exercice
 - Captures d'écran de GDB montrant les dumps mémoire
 - Sorties de Valgrind avant/après correction des fuites

Format : Archive TP2_NomPrenom.tar.gz

Commandes GDB utiles

```

x/Nxb addr    # N octets en hex
x/Nxw addr    # N mots (4 octets) en hex
x/Ncb addr    # N caractères
x/s addr      # String à partir de addr
p/x var       # Variable en hex
p/t var       # Variable en binaire

```